

Appunti Post pausa. Ore 11.20

Ragioneremo sulla nostra app in termini di stato - Loggato o non loggato.

src > app > test-logged > test-logged.html > button

```
1 <h1> Test API Loggato o non Loggato </h1>
2 <h2> Attualmente loggato:
3   {{user.email}} {{user.password}}
4 </h2>
5 <button (click)="doLogin()">Login</button>
6 /button>
```

Test API Loggato o non Loggato

Attualmente loggato: NESSUNA NESSUNA

```
1 import { HttpClient } from '@angular/common/http';
2 import { Component, inject } from '@angular/core';
3 import { FormsModule } from '@angular/forms';
4
5 interface User{
6   email:string;
7   password:string;
8 }
9
10 @Component({
11   selector: 'app-test-logged',
12   imports: [FormsModule],
13   templateUrl: './test-logged.html',
14   styleUrls: ['./test-logged.css'],
15 })
16 export class TestLogged {
17
18   // horror vacui. Non si fa questo in un componentttttt!
19   http = inject(HttpClient);
20   user:User = {email:"NESSUNA", password:"NESSUNA"};
21
22
23   doLogin():void{
24     this.http.post("http://localhost:8080/sbb/api/users/login", {"username":"llauro", "password":"pippo"})
25       .subscribe(json=>{
26         console.log(json);
27         const token = (json as any).token;
28         console.log("Token ricevuto "+token);
29         // jwt = intestazione.corpo.firma
30         let parts = token.split(".");
31         let hdv = parts[1]; // c'è l'utente (subject) ci sono i suoi claims
```

Il token non viene salvato in una variabile angular. Dopo che ho fatto login, me lo prendo anche in angular ma non basta.

(Togliamo la password, ovviamente non la devo inviare così)

src > app > test-logged > test-logged.html > user > username

```
5 interface User{
6   firstName:string;
7   lastName:string;
8   email:string;
9 }
10
11 @Component({
12   selector: 'app-test-logged',
13   imports: [FormsModule],
14   templateUrl: './test-logged.html',
15   styleUrls: ['./test-logged.css'],
16 })
17 export class TestLogged {
18
19   // horror vacui. Non si fa questo in un componentttttt!
20
21   http = inject(HttpClient);
22   user = signal<User>({firstName:"", lastName:"", email:""});
23
24   doLogin():void{
```

```

22
23
24 doLogin():void{
25     this.http.post("http://localhost:8080/sbb/api/users/login", {"username":"llauro", "password":"pippo"})
26     .subscribe(json=>{
27         console.log(json);
28         const token = (json as any).token;
29         console.log("Token ricevuto "+token);
30         // jwt = intestazione.corpo.firma
31         let parts = token.split(".");
32         let body = parts[1]; // c'è l'utente (subject), ci sono i suoi claims
33         console.log('Token Body:', body);
34         // traduce il codice che ci arrivato dalla codifica che ci ha mandato a testo normale
35         let translated = atob(body);
36         console.log('Decoded Token Body:', translated);
37         const payload = JSON.parse(translated);
38         console.log('Decoded Token Payload:', payload);
39
40         this.user.set({
41             firstName:payload.firstName,
42             lastName:payload.lastName,
43             email:payload.email
44         });
45     });
46 }
47
48 doTest():void{
49     this
50

```

Dove va salvato il token? Il token va salvato in localStorage.

LocalStorage è un piccolo db locale in js. È una mappa con chiave string e valore stringa.

```

        email:payload.email
    });

    // è una MAPPA con chiave string e valore string
    localStorage.setItem("user", JSON.stringify(this.user()));

}

doTest():void{
    this

```

LEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

::\Users\Ferdinando\Desktop\securitytest> ng serve

```

PS C:\Users\coral> Get-ChildItem Env:
Name Value
----
ALLUSERSPROFILE C:\ProgramData
APPDATA C:\Users\coral\AppData\Roaming
CommonProgramFiles C:\Program Files\Common Files
CommonProgramFiles(x86) C:\Program Files (x86)\Common Files
CommonProgram6432 C:\Program Files\Common Files
COMPUTERNAME PCFAMILY
ComSpec C:\WINDOWS\system32\cmd.exe
DriverData C:\WINDOWS\system32\Drivers\DriverData
HOME C:\Users\coral
HOMEPATH \Users\coral
LOCALAPPDATA C:\Users\coral\AppData\Local
LOGONSERVER \\PCFAMILY
NUMBER_OF_PROCESSORS 12
OneDrive C:\Users\coral\OneDrive
OS Windows_NT
Path C:\Program Files\Oracle\Java\javapath;C:\WINDOWS\system32;C:\WINDOWS;C:\...
PATHEXT .COM; .EXE; .BAT; .CMD; .VBS; .VBE; .JS; .JSE; .WSF; .WSH; .MSC; .CPL
PROCESSOR_ARCHITECTURE AMD64
PROCESSOR_IDENTIFIER Intel64 Family 6 Model 158 Stepping 10, GenuineIntel
PROCESSOR_LEVEL 6
PROCESSOR_REVISION 9c8a
ProgramData C:\ProgramData
ProgramFiles C:\Program Files
ProgramFiles(x86) C:\Program Files (x86)
Program6432 C:\Program Files
PSModulePath C:\Users\coral\Documents\WindowsPowerShell\Modules;C:\Program Files\WindowsPowerShell...
PUBLIC C:\Users\Public
SESSIONNAME Console
SystemDrive C:
SystemRoot C:\WINDOWS
TEMP C:\Users\coral\AppData\Local\Temp
TMP C:\Users\coral\AppData\Local\Temp
USERDOMAIN PCFAMILY
USERDOMAIN_ROAMINGPROFILE PCFAMILY
USERNAME coral
USERPROFILE C:\Users\coral
windir C:\WINDOWS
WSLENV WT_SESSION;WT_PROFILE_ID;
WT_PROFILE_ID {61c5b8bd-c2c6-5271-96e7-089a87f44bdf}
WT_SESSION 782ef01d-4074-4b8d-89f6-dfcdedf1d39e
ZES_ENABLE_SYSMAN 1

```

PS C:\Users\coral> |

Prima di fare la richiesta di login devo controllare se ho un utente in LocalStorage.

LOGOUT

Fare il logout vuol dire svuotare il LocalStorage

```
src > app > test-logged > test-logged.html > button
Go to component
1 <h1> Test API Loggato o non Loggato </h1>
2 <h2> Attualmente loggato:
3   {{user().email}} {{user().firstName}} {{user().lastName}}
4 </h2>
5 <button (click)="doLogin()">Login</button>
6 <button (click)="doLogout()">Logout</button>
7   doLogout (method) Testlogged.ds
8
```

```
doLogout():void{
  localStorage.removeItem("user");
  this.user.set({firstName:"", lastName:"", email:""});
}

doLogin():void{
```

```
src > app > test-logged > test-logged.html > button
Go to component
1 <h1> Test API Loggato o non Loggato </h1>
2 <h2> Attualmente loggato:
3   {{user().email}} {{user().firstName}} {{user().lastName}}
4 </h2>
5 <button (click)="doLogin()">Login</button>
6 <button (click)="doLogout()">Logout</button>
7 <button (click)="doTest()">Test {{result()}}</button>
8
9
10
```

Quello che devo salvare in local storage è il token e quando farò una richiesta, in berear, dopo lo spazio ..

```
4   }
5   }
6
7   doTest():void{
8     this
9     .http
10    .get<any>({
11      url: "http://localhost:8080/sbb/api/rooms/free/1/2026-02-27/2026-02-28",
12      headers: { "Authorization": "Bearer "+localStorage.getItem("token") }
13    })
14    .subscribe({
15      next: (json) => this.result.set("Loaded "+json.length+" rows"),
16      error: (error) => this.result.set(error.message)
17    });
18  }
19
20
21
22
23
```

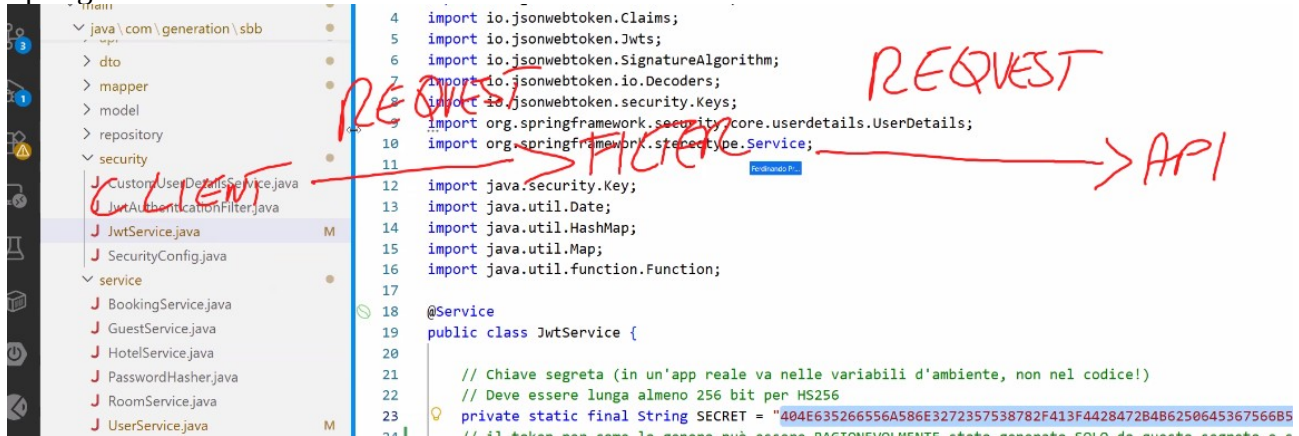


La parte pubblica è il login. Tutte le le altre api sono private.

Il JWT-SERVICE genera il token, ma dov'è che spring lo richiede?

Abbiamo detto che l'utente deve inviare il token. La prima volta fa il login, lo riceve, dopodiché lo memorizza e ogni chiamata successiva lo reinvia. Il cliente deve fornire le credenziali ogni volta.

Spring lavora con un sistema di filtri.



La API non è esposta direttamente. Spring lo nasconde, ma la API non è esposta direttamente (nessuna). Ci sono sempre dei filtri in mezzo. Ci si aspetta che quando lavoriamo con spring

creiamo dei filtri di accesso. questo è solo UNO Dei tanti filtri possibili

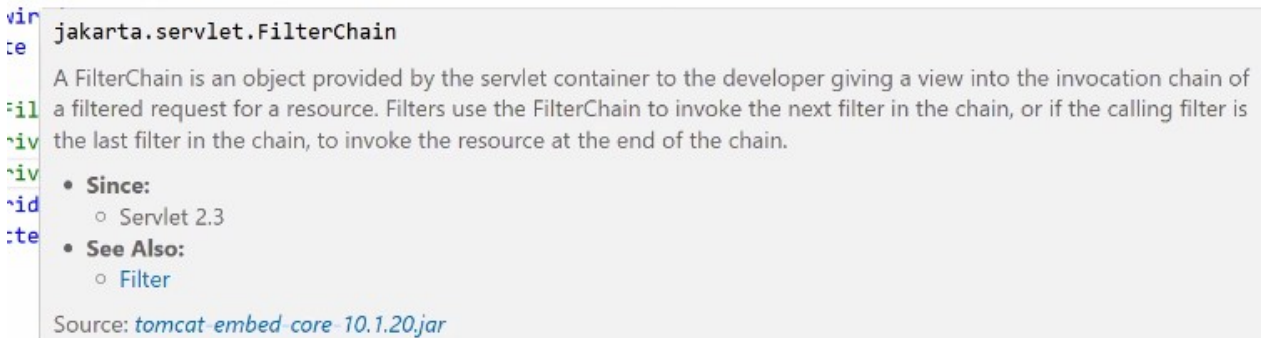
potrebbero essercene dozzine, infatti si parla di filterChain, catena dei filtri, in questo filtro io

analizzo la request che mi arriva e decido se riconoscere l'utente o non riconoscerlo se passare

avanti nella catena dei filtri, se ce ne sono altri senza impostare un utente riconosciuto o se invece

riconoscerlo devo gestire la request, devo gestire i suoi header.

mi serve jwtService, devo lavorare con JWT, col Json web Token, devo preparare gli eventuali dati per l'utente loggato.



`doFilterInternal`: esecuzione del filtro vero e proprio

arriva la request (piena), arriva la response (probabilmente vuota)

arriva la filterChain, cioè l'elenco dei filtri eseguiti e ancora da eseguire.

// prendo la request, nuda, per come è. Non prendo il suo JSON, la prendo TUTTA // header e corpo

/* esempio: GET /sbb/api/rooms/free/1/2026-02-27/2026-02-28 HTTP/1.1 Host: localhost:8080

Accept: application/json Authorization: Bearer eyJhbGciOiJIUzI1NiJ9... */

// prendo il valore dell'header Authorization final String authHeader =

request.getHeader("Authorization"); final String jwt; final String username;

// non c'è l'header Authorization, non prova neanche ad autenticarsi OPPURE // non usa "Bearer"

che è richiesto per jwt. IO NON TI AUTENTICO, io passo al prossimo filtro if (authHeader == null

|| !authHeader.startsWith("Bearer ")) { filterChain.doFilter(request, response); // passa avanti nella

catena dei filtri return; }

```

// jwt: rimuovo i primi sette caratteri
jwt = authHeader.substring(7);
// estraggo lo user name? come? Non mi interessa... lo estraggo
username = jwtService.extractUsername(jwt);

// se ho trovato uno username nel jwt e il mio SecurityContext è ancora vuoto...
// ma che perdindirindina è un SecurityContextHolder?? CHE DIAVOLO È??

// il SecurityContext è il contesto in cui Spring memorizza le autenticazioni
// cioè l'utente loggato. Il mio scopo è registrare l'autenticazione dell'utente
// nel context, per permettere alla API di essere eseguita.
// NON è il CONTEXT DELLA DEPENDENCY INJECTION, è un security context, sono dati di
autenticazione

if (username != null && SecurityContextHolder.getContext().getAuthentication() == null)

if (username != null && SecurityContextHolder.getContext().getAuthentication() == null)
{
// carico i dettagli dell'utente
UserDetails userDetails = this.userDetailsService.loadUserByUsername(username);

// vedo se quadrano col token.
if (jwtService.isValid(jwt, userDetails))
{
UsernamePasswordAuthenticationToken authToken = new
UsernamePasswordAuthenticationToken(
userDetails,
null,
userDetails.getAuthorities()
);

authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

// registro una autorizzazione e la metto nel security context
// e poi farò filterChain.doFilter... e passerò al prossimo ma il security context è stato
// riempito, e io sono LOGGATO per QUESTA request. Solo per questa.
SecurityContextHolder.getContext().setAuthentication(authToken);
}
}

filterChain.doFilter(request, response);

il security context verrà utilizzato dalla security configuration // che cercherà se siamo loggati
oppure no.

```

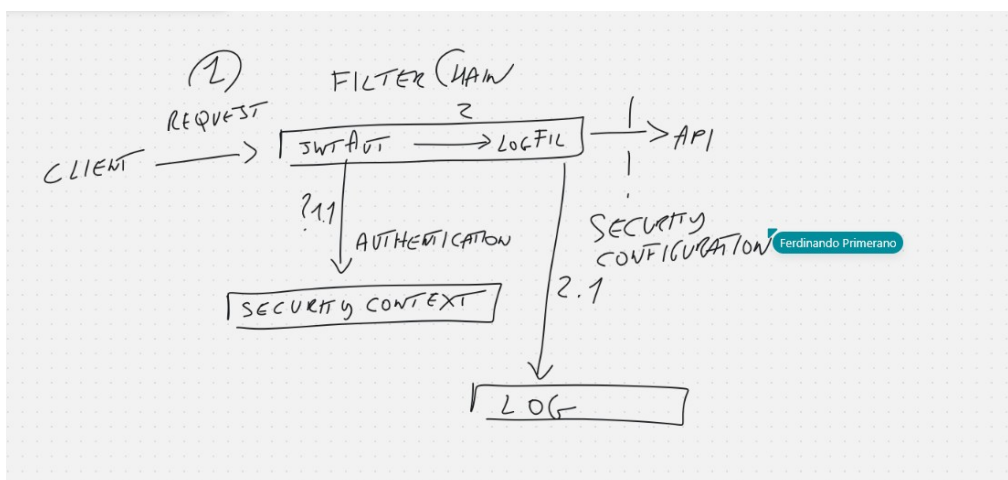
Il Filter Chain di Spring Security

Il diagramma illustra il flusso che una richiesta HTTP percorre all'interno del backend prima di raggiungere le API applicative. Quando il client invia una request, essa non arriva direttamente agli endpoint, ma viene intercettata da una struttura denominata Filter Chain, ovvero una catena di filtri che la processano in sequenza.

Il primo filtro incontrato è il `JwtAuthFilter`, responsabile dell'autenticazione. Questo filtro legge il token JWT presente nell'header della richiesta, ne verifica la firma e, se il token è valido, popola il `SecurityContext` con le informazioni di autenticazione dell'utente. Il `SecurityContext` è il contenitore interno di Spring Security che mantiene l'identità dell'utente corrente per tutta la durata dell'elaborazione della richiesta. Questa operazione è indicata nel diagramma come passo ?1.1, a sottolineare che si tratta di un passaggio condizionale: avviene solo se il token è presente e valido.

Il secondo filtro nella catena è il `LogFilter`, che si occupa della registrazione delle richieste in ingresso. A sua volta, questo filtro si appoggia alla `SecurityConfiguration` per determinare quali regole di accesso applicare: è qui che vengono definite le policy che stabiliscono quali endpoint sono pubblici e quali richiedono autenticazione. In base a queste regole, il `LogFilter` produce una voce nel log, indicato nel diagramma come passo 2.1.

Solo al termine di questo processo, se tutti i controlli hanno avuto esito positivo, la richiesta viene inoltrata all'API di destinazione. Il percorso completo è quindi: client → `JwtAuthFilter` → `SecurityContext` → `LogFilter` → `SecurityConfiguration` → Log → API.



```

9  import org.springframework.security.web.SecurityFilterChain;
10 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
11
12 @Configuration
13 @EnableWebSecurity
14 public class SecurityConfig {
15
16     @Autowired
17     private JwtAuthenticationFilter jwtAuthFilter;
18
19     @Bean
20     public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
21         http
22             .csrf(csrf -> csrf.disable()) // Disabilita CSRF per le API REST
23             .authorizeHttpRequests(auth -> auth
24                 .requestMatchers(...patterns: "/sbb/api/users/login").permitAll() // Assicura che il login sia sempre
25                 .anyRequest().authenticated() // A parte il login, tutto è autenticato.
26             )
27             .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
28             .addFilterBefore(jwtAuthFilter, beforeFilter: UsernamePasswordAuthenticationFilter.class);
29         return http.build();
30     }
31 }

```

La Classe SecurityConfig in Spring Security

La classe `SecurityConfig` è il punto centrale in cui vengono definite tutte le regole di sicurezza dell'applicazione backend. È annotata con `@Configuration` e `@EnableWebSecurity`, il che indica a Spring che deve essere trattata come una sorgente di configurazione e che deve abilitare il modulo di sicurezza web del framework.

Il metodo principale è `securityFilterChain`, che riceve un oggetto `HttpSecurity` e restituisce una `SecurityFilterChain`, ovvero la catena di filtri che verrà applicata ad ogni richiesta in ingresso. La configurazione si articola in quattro istruzioni fondamentali.

La prima disabilita la protezione CSRF tramite `csrf.disable()`. Questa scelta è appropriata per le API REST, poiché il meccanismo CSRF è pensato per applicazioni che utilizzano sessioni e cookie, mentre in un'architettura stateless basata su JWT la protezione viene garantita dalla firma del token stesso.

La seconda definisce le regole di autorizzazione tramite `authorizeHttpRequests`. Viene stabilito che l'endpoint `/sbb/api/users/login` è pubblico e accessibile senza autenticazione tramite `permitAll()`, poiché è proprio da lì che l'utente ottiene il token. Qualsiasi altra richiesta, invece, richiede obbligatoriamente che il chiamante sia autenticato, come espresso da `anyRequest().authenticated()`.

La terza configura la gestione delle sessioni impostando la policy `STATELESS`. Questo significa che il server non crea né mantiene alcuna sessione HTTP per i client: ogni richiesta è completamente autonoma e deve portare con sé il token per dimostrare la propria identità, coerentemente con il modello JWT.

La quarta inserisce il filtro personalizzato `JwtAuthenticationFilter` nella catena tramite `addFilterBefore`, posizionandolo prima del filtro standard `UsernamePasswordAuthenticationFilter`. Questo garantisce che il token JWT venga

verificato prima che Spring Security tenti qualsiasi altro tipo di autenticazione basata su username e password.

`SecurityConfig` è la classe che dice a Spring Security **chi può fare cosa**.

Disabilita CSRF perché non serve con JWT, stabilisce che solo il login è pubblico e tutto il resto richiede autenticazione, imposta il server come stateless perché non usa sessioni, e inserisce il filtro JWT nella catena prima di qualsiasi altro controllo.

```
return http.build();

// 24 API sempre raggiungibile login
// 25: tutte le altre request richiedono autenticazione
// 28 l'eventuale autenticazione sarà prodotta dal jwtAuthFilter
//
}
```

`@Bean` → Indica a Spring che questo metodo produce un componente gestito dal container, necessario per definire la catena dei filtri. In Spring Boot, la scelta tra `@Service`, `@Component` e `@Repository` non cambia il modo in cui Spring crea l'oggetto (il Bean), ma serve a dare un **significato semantico** preciso all'interno della tua architettura.

1. @Component

È l'annotazione generica "madre". Indica a Spring: "Questa è una classe gestita da te, creane un'istanza (Bean) e caricala nel context".

- **Uso nel tuo caso:** Il tuo `JwtAuthenticationFilter` dovrebbe essere annotato con `@Component`, poiché è un pezzo di logica infrastrutturale che deve essere "iniettato" nella catena dei filtri.

2. @Service

È una specializzazione di `@Component` usata per la **Business Logic** (logica applicativa).

- **Significato:** Indica che la classe contiene operazioni di calcolo, coordinamento o manipolazione dati.
- **Uso nel tuo caso:** Il tuo `JwtService` deve usare questa annotazione. Come hai scritto nei commenti, il suo compito è estrarre lo username o validare il token, operazioni che rientrano perfettamente nel concetto di "servizio".

3. @Repository

Specializzazione dedicata allo **strato di persistenza** (accesso ai dati).

- **Significato:** Oltre a creare il Bean, abilita la traduzione automatica delle eccezioni del database (es. SQL) in eccezioni specifiche di Spring.
- **Uso nel tuo caso:** Se il tuo `UserDetailsService` deve andare a cercare l'utente sul database (come accennato nei tuoi commenti sull'utente loggato), chiamerà una classe (es. `UserRepository`) annotata proprio con `@Repository`.

Si usa `@Bean` principalmente in tre scenari, proprio come nel tuo progetto di Security:

1. **Classi di terze parti:** Se vuoi trasformare in un Bean una classe che non hai scritto tu (es. `BCryptPasswordEncoder`), non puoi andare a scriverci sopra `@Component`. Devi creare un metodo `@Bean` che la istanzi.
2. **Configurazione personalizzata:** Quando la creazione dell'oggetto richiede logica complessa o parametri specifici.
3. **Disaccoppiamento:** Quando vuoi definire un'interfaccia come Bean, ma decidere l'implementazione in un file di configurazione centrale.

Guard → sono state impostate. Il frontend prevede di studiare gli interceptor e le guardie.

```
rc > app > TS room-service.ts > RoomService > findFreeRoomsForHotel
13 export class RoomService {
19   public findFreeRoomsForHotel(id:number, from:any, to:any):Observable<RoomCategory[]>{
20
21     console.log(from);
22     console.log(to);
23
24
25     if(from.toISOString()
26       from = from.toISOString().split("T")[0];
27     if(to.toISOString()
28       to = to.toISOString().split("T")[0];
29
30     let url = this.freeRoomsUrl+id+"/"+from+"/"+to;
31     return this.http.get<RoomCategory[]>(url);
32   }
33 }
34
35 }
```

```
let url = this.freeRoomsUrl+id+"/"+from+"/"+to;
// normalmente dovrei specificare gli header con Bearer e JWT...
// per evitare di farlo sempre c'è qualcosa di simile ai filter in Spring
// ma qui si chiamano interceptor
// il filter di spring lavora sul server sulla request che arriva alle api
// l'interceptor di angular lavora sulla request prima che venga inviata al server
// il filter può modificare la request o fare altre azioni sul server
// l'interceptor serve tendenzialmente a MODIFICARE LA REQUEST
return this.http.get<RoomCategory[]>(url);
```

```
0 let authReq = req;
1
2 // --- 1. AGGIUNTA DEL TOKEN (LOGICA IN USCITA) ---
3 if (token) {
4   // Cloniamo la richiesta originale per iniettare l'header Authorization
5   authReq = req.clone({
6     setHeaders: {
7       Authorization: `Bearer ${token}`
8     }
9   });
0 }
```

```

rc > app > TS app.config.ts > appConfig > providers
1  import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
2  import { provideRouter, withComponentInputBinding } from '@angular/router';
3
4  import { routes } from './app.routes';
5  import { provideHttpClient, withInterceptors } from '@angular/common/http';
6  import { authInterceptor } from './interceptors/authInterceptors';
7
8  export const appConfig: ApplicationConfig = {
9    providers: [
10     provideBrowserGlobalErrorListeners(),
11     provideRouter(routes, withComponentInputBinding()),
12     provideHttpClient(withInterceptors([authInterceptor])),
13   ]
14 };
15

```

alcune api sono private altre pubbliche. Cosa usiamo per autenticarmi sul server, il filter. Cosa usa per proteggere le api il security le api. Per evitare di mandare sempre il token uso un interceptor.